

From Delaunay to Voronoi

Now that we've completed our DT generation algorithm, the Voronoi diagram generation algorithm is within our grasp. We can turn a set of points into a Voronoi diagram by following this algorithm:

1. Find the DT of a set of points.
2. Take the circumcenter of every triangle in the DT.
3. Draw lines connecting the circumcenters of all triangles in the DT that share an edge.

We already know how to do step 1 (we did it in the previous section), and we can accomplish step 2 with the `triangle_to_circumcenter()` function. So the only thing we need is a code snippet that can accomplish step 3.

The code we write for step 3 will live in our plotting function. Remember that we pass a set of triangles and circumcenters to that function as its inputs. Our code will need to create a collection of lines connecting circumcenters. But it will not connect all of the circumcenters, only those from triangles that share an edge.

We're storing our triangles as collections of points, not edges. But it's still easy to check whether two of our triangles share an edge; we just check whether they share exactly two points. If they share only one point, then they have vertices that meet but no common edge. If they share three points, they are the same triangle and so will have the same circumcenter. Our code will loop through every triangle, and for each triangle, it will loop through every triangle again, and check the number of points that the two triangles share. If the number of common points is exactly two, then it will add a line between the circumcenters of the triangles in question. The lines between the circumcenters will be the boundaries of our Voronoi diagram. The following code snippet shows how we'll loop through triangles, but it's part of a larger plotting function, so don't run it yet:

```
--snip--
for j in range(len(triangles)):
    commonpoints = 0
    for k in range(len(triangles[i])):
        for n in range(len(triangles[j])):
```